NATIONAL INSTITUTE OF TECHNOLGY WARANGAL
Department Of Computer Science and Engineering

PROJECT-BASED LAB REPORT

AI-Powered Secure Lexical Analyzer

A Hybrid Rule-Based and Deep Learning Approach to Token-level Security Analysis in Python Source Code

Course: Compiler Design

Department Of Computer Science and Engineering

National Institute of Technology Warangal

Submitted to: Prof. E. Suresh Babu, Associate Professor

Academic Year: 2025-2026

By

Sai Pranathi Vanaparthi

24CSB0A63

Abstract

Source Code security analysis is a critical challenge in modern software development, particularly when users embed malicious constructs within syntactically valid programs. Unicode-based obfuscation, injection attacks, and high-entropy payloads, represent a class of threats that traditional compilers and static analysis tools fail to detect such issues, as these tools prioritize syntactic correctness over security semantics.

This work presents an AI-Powered Secure Lexical Analyzer designed to detect token level threats in the Python source code. The system incorporates a regex-based lexer with a Unicode recovery engine, a 45-dimensional security feature extractor, a Bidirectional Long Short-Term Memory (BiLSTM) neural network, a rule-based validator, and a taint tracking engine within a unified hybrid decision framework.

The analyzer classifies every token in the Python source file into one of the three security classes defined: SAFE, SUSPICIOUS, MALICIOUS. Experimental evaluation demonstrates detection of homoglyph attacks, zero-width character injection, SQL injection patterns, shell command injection, exec/eval based code injection, and Base-64-encoded payloads. The hybrid architecture combines rule-based prediction with the contextual generalization of BiLSTM, with 60% weight given to rule based and 40% given to AI, achieves superior weighted recall on MALICIOUS tokens compared to a purely feedforward baseline. The system also produces structured explanatory output that identifies the specific reason why that token is flagged, enabling actionable developer feedback.

The key contributions of my project are:

1. A novel Unicode recovery pipeline that maps confusable characters and removes invisible characters before classification.
2. A 45-dimensional feature vector that captures Unicode, entropy, recovery, encoding, and contextual properties.
3. A sliding window BiLSTM that classifies tokens in a sequential context rather than in isolation.
4. A taint-like engine that tracks user-controlled data flow from source functions to dangerous API sinks.

1.Introduction

1.1 Context and Importance

Modern software supply chains are increasingly threatened by source code attacks that exploit the gap between syntactic validity and security safety. An attacker who can inject Python code into a build pipeline, code review tool, or AI-assisted development environment can embed malicious constructs that pass syntactic checks while performing harmful operations at runtime. Unicode (which helps to support multiple languages) allows different characters from different scripts to look exactly the same. Attackers misuse this by creating fake names or identifiers that look identical to real ones but are actually different underneath. They also use tricks like invisible characters (Zero-width), bidirectional overrides, encoded text to hide harmful content. So even if something looks normal, it might secretly contain malicious code.

The lexical analysis phase is the first step in a compiler. In this stage, the source code is broken into small pieces called tokens. A lexer incorporating security analysis can therefore intercept threats before even it progresses to parsing, semantic analysis, or execution. This project extends a classical lexer with security intelligence, combining the interpretability of rule-based systems with the generalization capability of deep learning.

Existing tools address code security at different phases of the compilation. GCC and Clang perform static analysis at the intermediate representation level, while the Python-specific tools such as Bandit and Semgrep operate on the Abstract Syntax Tree (AST). Whereas transformer-based models like CodeBERT generalize well to semantic vulnerability patterns. But, all of these tools operate after the Python parser has normalized Unicode, making them blind to class of lexical phase threats described in Section 3.

1.2 Research Question

The main research question of this project is: Can a hybrid lexer, combining rule-based token validation with a Bidirectional LSTM trained on security labelled token features, detects malicious token patterns in the Python source

code, including Unicode Obfuscation, injection attacks, and encoded payloads, while providing human-readable explanations for each detection.

1.3 Methodology Overview

The system processes Python source code through 6 phases:

1. Unicode pre-scan and recovery.
2. Regex-based tokenisation and enrichment.
3. 45-feature extraction per token.
4. BiLSTM sequence classification.
5. Rule-based validation and taint tracking.
6. Hybrid decision fusion with explainability output.

Each phase is described in detail in Section 5.

1.4 Original Contributions

- A Unicode Recovery Engine that normalizes, maps confusable characters and strips invisible zero width characters before token classification, preserving the original token for audit.
- A 45-dimensional feature vector extending the standard 34-dimensional baseline with recovery, encoding, fuzzy keyword and neighbouring density features.
- A BiLSTM classifier operating on sliding windows of 5 tokens, which enables sequence aware token detection.
- A taint-lite engine that tracks data flow user-controlled source functions to dangerous API sinks within a token stream.
- A structured explanatory output that maps every flagged token to a human readable list of actual reasons.

1.5 Report Structure

Section 2	Background Concepts.
Section 3	Surveys related works.
Section 4	Architectural Design.
Section 5	Implementation.
Section 6	System Evaluation.
Section 7	Conclusion & Future Direction.

2. Background Information

2.1 Lexical Analysis

Lexical Analysis is the first phase of a compiler. The lexical analyzer is also referred as lexer. A lexer reads a stream of characters and produces a sequence of tokens, where each token is a pair (token-type, value). Token types include IDENTIFIER, KEYWORDS, INTEGER, FLOAT, STRING, OPERATOR and DELIMITER. Classical traditional lexers are implemented using deterministic finite automata derived from regular expressions. This project uses a compiled master regex pattern that matches all token types simultaneously, selecting the type based on which named group in the pattern match.

2.2 Unicode and Homoglyph Attacks

The Unicode Standard (IEC 10646) assigns a unique code point to every character across all writing systems. Many characters from languages like Cyrillic and Greek look exactly like normal English (Latin) letters, even though they are actually different. For example, the Cyrillic small letter 'a' (U+0430) is visually identical to the Latin small letter 'a' (U+0061) in most typefaces. An attacker can create an identifier 'admin' using Cyrillic characters that appears identical to the legitimate identifier 'admin' but occupies a different memory location at runtime.

Zero-width characters (U+200B, U+200C, U+200D) and bidirectional override characters (U+202A through U+202E) are invisible in most editors and rendering environments, yet they alter string equality comparisons and can be used to smuggle content past text-based security filters.

NFKC (Normalisation Form KC: Compatibility Decomposition followed by Canonical Composition) is the Unicode normalisation form recommended for security-sensitive identifier comparisons. It maps compatibility characters to their canonical equivalents, eliminating many confusable distinctions.

2.3 Shannon Entropy

Shannon entropy H is a measure of the average information content of a given source. For a string of characters, the entropy is calculated as:

$$H = -\Sigma\, p(c) \times \log_2(p(c))$$

Where p(c) is the probability of character c in the string. High-entropy strings contain more randomness and are the characteristics of encoded payloads, cryptographic keys, and obfuscated code. The system uses character entropy, bigram entropy, and trigram entropy as its features.

2.4 Bidirectional LSTM

A Long Short-Term Memory (LSTM) network is an improved version of Recurrent Neural Network (RNN) which is designed to capture long-term dependencies in sequential data. It maintains a memory cell to store the information over time, solving the limitations of traditional RNNs.The LSTM architecture was originally proposed by Hochreiter and Schmidhuber. The memory cell in LSTM architecture is controlled by 3 gates: input gate, forget gate, and output gate. This allows LSTM to selectively retain or discard information as it flows through the network which allow the LSTM to learn the neural dependencies.

A bidirectional Long Short-Term memory (BiLSTM) model extends the present capabilities of the LSTM further by processing the input sequence in both forward and backward directions, allowing it to capture both the past and future context. In this project, a sliding window of size 5 tokens is fed to the BiLSTM, so the classification of a token is informed by two tokens on each side.

2.5 Taint Analysis

Taint analysis is a program analysing technique that tracks the flow of untrusted data through a program. A taint source is a program is a program point where external, potentially suspicious or malicious controlled data enters the program such as user input or HTTP request parameters. A taint sink is a program point where the data can be dangerous if it is user controlled such as database query or a shell command. Full taint analysis requires dependency graph of program, but the taint-lite engine in this project approximates it using token stream pattern matching.

2.6 Hybrid Classification

The hybrid classifier combines two or more classification mechanisms, typically a rule-based system and a machine learning model, to leverage the interpretability and precision of rules with the generalization capacity of the learning models. The combination is a weighted average of class probability vectors, including override rules for high confidence rule detections.

3. State of the Art

3.1 Compiler-Based Security Analysis

GCC and Clang provide the basic security warnings through their static analysis passes. Both of the compilers detect buffer overflows, use-after-free patterns, and integer overflows at the intermediate representation level of the compilation. GCC 12 has introduced -fnalyzer, a path sensitive inter-procedural static analysis pass. Whereas Clang's clang-tidy provides a plugin architecture for custom lint checks. Neither of these compilers addresses Unicode-base identifier obfuscation, homoglyph attacks, or token-level entropy analysis. The lexical analysis phase in both of the compilers performs no security classification, treating all the syntactically valid identifiers as safe and equivalent regardless of their Unicode composition. This gap motivates analysis at a phase that precedes all compiler passes (the lexer).

3.2 Python-Specific Static Analysis Tools

Bandit is the most widely used Python security analysis tool. It operates on the concept of Abstract Syntax Tree (AST) and applies a set of rules that detect common security issues like use of exec(), weak cryptographic algorithms, hardcoded passwords, and SQL query construction using string formatting. Bandit does not perform lexical level analysis and therefore cannot detect Unicode obfuscation, zero-width character injection, or homoglyph attacks, as the Python parser normalizes identifiers before AST construction.

Semgrep provides a pattern matching engine build over ASTs with support for dataflow analysis. It allows users to define patterns in a domain specific language and supports taint propagation rules. Semgrep's strengths are in semantic patterns, its weakness is that it operates post parsing and therefore it inherits the same blind spot as Bandit with respect to lexical level Unicode attacks.

Pyflakes focuses on logical errors like undefined names, unused imports, and redefined variables. It does not perform any security analysis. Since Python normalizes identifiers before AST construction, any tool operating at or after parsing is architecturally incapable of detecting Unicode-based attacks.

3.3 Boucher and Unicode Security Research

Nicholas Boucher and Ross Anderson of Cambridge University have demonstrated that Unicode bidirectional control characters can be used to create a source that appears to be completely normal to human reviewers but executes differently often bypassing the security checks, they have named this attack as "*Trojan Source*". They have found that it affects some of the most high-level programming languages like C, C++, JavaScript, Python, Rust, Go, SQL, Bash. The subsequent patches were applied to compilers, but the underlying lexical trust assumption was not addressed fundamentally.

The Unicode Consortium has published the Unicode Security Mechanisms (Unicode Technical Report 39), which describes the potential security risks of using Unicode Standard, by mapping visually confusable characters to their canonical equivalents. The TR39 dataset contains over 10000 mappings, this implementation uses a set of 10-25 high frequency Cyrillic and Greek confusable.

This work directly motivates the Unicode recovery engine in Phase 1 of the proposed system.

3.4 Machine Learning for Code Security

CodeBERT (2020) is a pre-trained transformer model for programming languages, that is trained on the GitHub code. It has been applied to detect vulnerabilities by fine tuning on labelled vulnerability datasets. CodeBERT operates at the sub token level and it has strong performance on semantic vulnerability patterns but is not designed for token level security analysis, requires significant computational resources and does not address Unicode attacks.

Devign (2019) uses a gated graph based neural network over combines AST, control flow, flow graphs for detecting function level vulnerabilities in C. It demonstrates that graph structured demonstrations of code outperform sequence models for the detection of semantic vulnerability. However, its focus on C and function level granularity makes it inapplicable to token level Python analysis. Neither model operates at the lexical phase or addresses Unicode obfuscation, leaving token-level security analysis as an open problem.

3.5 Positioning This Work

This project occupies a distinct position in the landscape. This is the only system, to my knowledge that performs security analysis at the lexical phase of a Python compiler pipeline, before parsing Unicode identifiers. Its token level granularity allows to detect threats that AST based tools cannot detect. The BiLSTM architecture allows it to generalize beyond the exact match rules that characterize all the lexical security tools. Table 1 summarises all the comparison.

Feature	GCC/Clang	Bandit	Semgrep	CodeBERT	Devign	This Work
Unicode homoglyph detection	No	No	No	No	No	Yes
Zero-width char detection	No	No	No	No	No	Yes
Lexical-phase analysis	No	No	No	No	No	Yes
ML-based classification	No	No	No	Yes	Yes	Yes
Sequence context	No	No	No	Yes	Yes	Yes
Token recovery engine	No	No	No	No	No	Yes
Taint tracking	Partial	No	Yes	No	Partial	Lite
Explainability output	No	No	No	No	No	Yes
Python-specific	No	Yes	Yes	Yes	No	Yes

Table 1: Comparison of this work with related tools

4. Design of Solution

4.1 System Architecture Overview

The system is structured as a six-phase pipeline. Each phase is a self-contained module with defined inputs and outputs, enabling testing and extension. Figure 1 presents the pipeline overview.

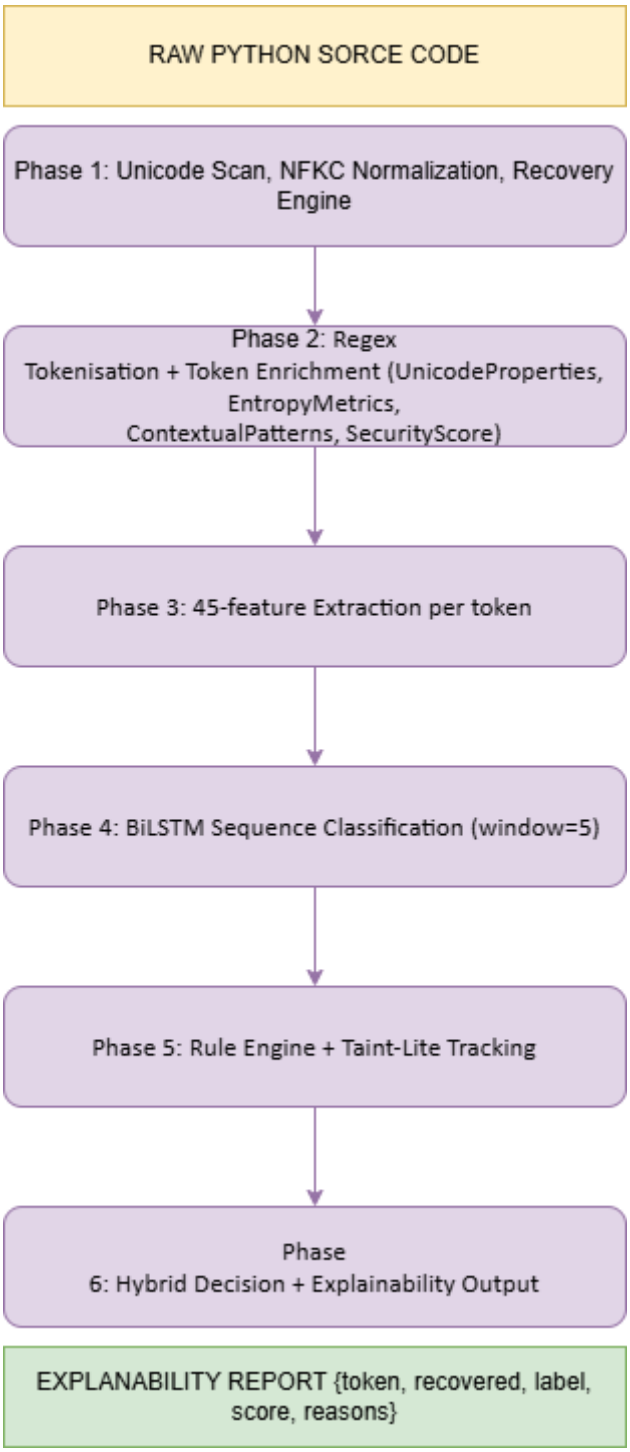


Figure 1: System *pipeline overview*

4.2 Token Data Model

Every token in the stream is represented as a SecureToken object that contains four core fields (raw_value, token_type, line, column) and four enrichment fields populated after lexing: UnicodeProperties, EntropyMetrics, ContextualPatterns, and SecurityScore. This design allows the lexer to operate at full speed on the first pass and defer expensive security analysis to the enrichment pass.

4.3 BiLSTM Architecture

The neural classifier uses two inputs, a sequence of 34-dimensional legacy feature vectors over a window of 5 tokens (processed by BiLSTM), and a single 45-dimensional rich feature vector for the centre token (processed by a feedforward projection head). The two representations are concentrated to a three-layer feedforward classifier producing logits over three classes.

Component	Dimensions	Description
Input projection	34 → 64	Linear + LayerNorm + ReLU + Dropout (0.3)
BiLSTM	64 → 256 (128×2)	2 layers, bidirectional, batch-first
Sequence pooling	256 → 256	Mean pooling over seq_len dimension
Feature projection	45 → 64	Linear + LayerNorm + ReLU + Dropout (0.15)
Concatenation	256 + 64 = 320	Sequence vector + feature vector
Classifier head	320 → 128 → 64 → 3	ReLU + Dropout between layers

Component	Dimensions	Description
Output	3	Logits for SAFE, SUSPICIOUS, MALICIOUS

Table 2: BiLSTM model architecture

4.4 Hybrid Decision Design

The hybrid decision combines rule-based and ML predictions using a weighted average of class probability vectors, with a rule weight of alpha = 0.6. Three override rules take precedence over the weighted average:

1. If a token contains zero-width characters (Unicode risk HIGH), it is escalated to MALICIOUS regardless of the ML prediction;

2. If a token's character entropy exceeds 4.5, it is escalated to SUSPICIOUS if the hybrid prediction is SAFE;

3. If taint analysis identifies the token as a taint sink, it is escalated to MALICIOUS class. These overrides encode domain knowledge that is always correct, preventing the ML model from overriding clear rule-based evidence.

4.5 Dual Analysis Design

Each token is classified twice. Once with its original value and then once with its recovered (Unicode normalized) value. The final class is the element wise maximum of the two probability vectors. This design ensures that a token which is fine and safe in its original form but reveals a dangerous keyword after Unicode recovery, it is still caught. For Example, a token 'exec' composed entirely of Cyrillic characters scores low on rule based features because it does not match the ASCII string ‘exec’, but after confusable mapping recovers to ‘exec’ and the recovered token prediction flags it correctly.

4.6 System Interface

The system exposes a web-based interface that allows users to paste Python source code and receive a per-token security analysis report. The interface provides five preset code samples — Clean Code, Homoglyph, SQL Injection, exec/eval, Taint Flow, and Zero-Width — that demonstrate the system's detection capabilities. Figure 2 shows the interface with the clean code sample loaded. Upon clicking Analyse, the backend invokes the full six-phase pipeline and renders the classification results on the right panel, displaying each flagged token's label, score, and reasons.

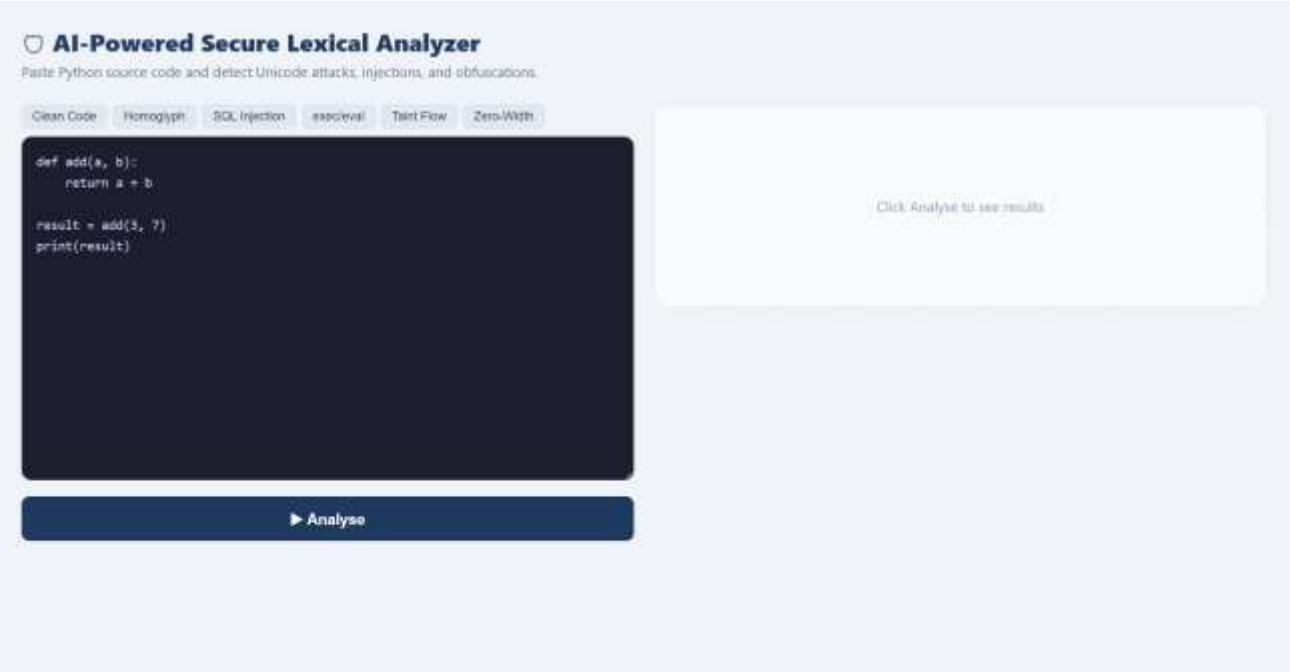






Figure 2: Web interface of the AI-powered Secure Lexical Analyzer

5. Realization of Solution

5.1 Project Structure

The project is organized into two primary modules. The W5_6 module contains the core lexer - lexer.py, token data model - token_models.py, grammar specification - token_grammar.py, and symbol_table - symbol_table.py. The ML_model module contains the recovery engine - recovery_engine.py, feature extractor and BiLSTM model – ai_model.py, dataset builder – build_dataser.py, training script – train_model.py, hybrid validator – hybrid_validator.py, and pipeline runner – run_secure_engine.py.

The following subsections present each phase of the pipeline in both prose and algorithmic pseudo code. Each algorithm maps directly to corresponding implementation.

5.2 Phase 1: Unicode Recovery Engine.

The RecoveryEngine class implements the recovery pipeline in four sequential steps which is applied to each token string. First, NFKC normalisation is applied using Python's unicodedata.normalize function. Second, confusable mapping replaces 34 Cyrillic and Greek code points with their Latin equivalents using a lookup table. Third, zero-width characters (U+200B, U+200C, U+200D, U+2060, U+FEFF) are removed by filtering. Fourth, bidirectional control characters (U+202A through U+202E, U+200E, U+200F) are removed. The original token is preserved; only the recovered copy is modified. A RecoveryResult data class records which transformations were applied and the normalised difference ratio.

```
# ML_model/recovery_engine.py — RecoveryEngine.recover() method

def recover(self, token: str) -> RecoveryResult:
    result = RecoveryResult(original=token, recovered=token, changed=False)
    t = token
    # Step 1: NFKC normalisation
    t_norm = unicodedata.normalize('NFKC', t)
    if t_norm != t:
        result.normalized = True
        result.log.append(f"NFKC: {repr(t)} -> {repr(t_norm)}")
        t = t_norm
    # Step 2: Confusable mapping (Cyrillic/Greek → Latin)
    t_conf, n_conf = self._map_confusables(t)
    if n_conf:
        result.confusables_mapped = n_conf
        result.log.append(f"Confusables mapped: {n_conf}")
        t = t_conf
```

```
    # Step 3: Zero-width character removal
    t_zws, n_zws = self._remove_chars(t, ZWS_SET)
    if n_zws:
        result.zws_removed = n_zws
        result.log.append(f"ZWS removed: {n_zws}")
        t = t_zws
    # Step 4: Bidi override removal
    t_bidi, n_bidi = self._remove_chars(t, BIDI_SET)
    if n_bidi:
        result.bidi_removed = n_bidi
        t = t_bidi
    result.recovered = t
    result.changed   = (t != token)
    return result
```

5.3 Phase 2: Token Enrichment

After tokenisation, every token is enriched by calling enrich() method, which populates four metadata objects. UnicodeProperties computes per-character statistics including non-ASCII count, zero-width count, homoglyph count, RTL character count, and script membership. EntropyMetrics computes Shannon entropy at the character, bigram, and trigram levels. ContextualPatterns inspects the surrounding token stream within a window of three tokens to detect SQL keywords, shell command patterns, Base64 patterns, dunder names, and builtin shadows. SecurityScore aggregates these properties into a single score in the range [0, 1] using a weighted sum, flagging tokens with score >= 0.5 as suspicious.

```
# W5_6/token_models.py — SecurityScore.compute()
@classmethod
def compute(cls, unicode_props, entropy, context) -> "SecurityScore":
    ss = cls()
    w  = ss.WEIGHTS
    total, reasons = 0.0, []
    if unicode_props.non_ascii_count > 0:
        total += w["non_ascii"]
        reasons.append(f"non-ASCII chars: {unicode_props.non_ascii_count}")
    if unicode_props.zero_width_count > 0:
        total += w["zero_width"]
        reasons.append(f"zero-width chars: {unicode_props.zero_width_count}")
    if entropy.is_high_entropy:
        total += w["high_entropy"]
        reasons.append(f"high entropy: {entropy.char_entropy:.3f}")
    if unicode_props.homoglyph_count > 0:
        total += w["homoglyph"]
        reasons.append(f"homoglyph chars: {unicode_props.homoglyph_count}")
    if context.precedes_exec_or_eval:
        total += w["exec_eval_context"]
        reasons.append("used near exec/eval")
    if context.matches_shell_pattern:
        total += w["shell_pattern"]
        reasons.append("matches shell pattern")
    ss.score       = min(total, 1.0)
    ss.reasons     = reasons
```

ss.is_suspicious = ss.score >= ss.threshold

return ss

5.4 Phase 3: 45-Feature Extraction

The extract_45_features function produces a 45-dimensional feature vector from a SecureToken object and its neighbours. The features are organised into seven groups as described in Table 3.

Indices	Group	Features
0-3	Token type	token_type (normalised), length, word count, is_numeric
4-9	Unicode	ZWS count, bidi count, confusable count, non-ASCII ratio, homoglyph count, script count
10-13	Entropy	char entropy, bigram entropy, trigram entropy, normalised entropy
14-18	Char ratios	digit ratio, uppercase ratio, special char ratio, lowercase ratio, underscore ratio
19-25	Recovery	recovery_changed flag, diff ratio, ZWS removed, confusables mapped, is_base64, decoded_keyword_score, recovered_entropy
26-30	Security	keyword score (raw), keyword score (recovered), fuzzy keyword score, dangerous API flag, SQL keyword flag
31-41	Context	nesting depth, follows_import, near_exec_eval, is_dunder, is_builtin_shadow, security score, is_suspicious, shell pattern, neighbour dangerous density, neighbour suspicious density, neighbour high-entropy density

Indices	Group	Features
42-44	Token type flags	is_IDENTIFIER, is_STRING, is_KEYWORD (one-hot)

Table 3: 45-dimensional feature vector groups

```
def extract_45_features(token, neighbor_tokens: Optional[List] = None) -> List[float]:
    raw = token.raw_value if token.raw_value else ""

    #  Recovery
    rec = _recovery_engine.recover(raw)
    features = [0.0] * 45
    # [0] Token type (normalised)
    features[0] = token.token_type.value / 50.0

    # [1] Length
    length = len(raw)
    features[1] = min(length / 100.0, 1.0)

    # [2] Word count (spaces + 1)
    features[2] = min((raw.count(' ') + 1) / 10.0, 1.0)

    # [3] Is numeric literal
    try:
        float(raw)
        features[3] = 1.0
```

```
except (ValueError, TypeError):
    features[3] = 0.0

# [4] Zero-width char count
features[4] = min(_count_zws(raw) / 5.0, 1.0)

# [5] Bidi char count
features[5] = min(_count_bidi(raw) / 5.0, 1.0)

# [6] Confusable char count
features[6] = min(_count_confusables(raw) / 10.0, 1.0)

# [7] Non-ASCII ratio
non_ascii = sum(1 for ch in raw if ord(ch) >= 128)
features[7] = non_ascii / max(length, 1)

# [8] Homoglyph count (from pre-computed unicode props if available)
if token.unicode:
    features[8] = min(token.unicode.homoglyph_count / 10.0, 1.0)
else:
    features[8] = features[6]   # fall back to confusable count

# [9] Script count (mixed scripts signal)
if token.unicode:
    features[9] = min(len(token.unicode.scripts_present) / 5.0, 1.0)
else:
```

```
        features[9] = 0.0

    # [10] Character entropy
    features[10] = min(_shannon_entropy(raw) / 5.0, 1.0)

    # [11] Bigram entropy
    bigrams = [raw[i:i+2] for i in range(len(raw) - 1)]
    features[11] = min(_shannon_entropy(''.join(bigrams)) / 5.0, 1.0) if bigrams
else 0.0

    # [12] Trigram entropy
    trigrams = [raw[i:i+3] for i in range(len(raw) - 2)]
    features[12] = min(_shannon_entropy(''.join(trigrams)) / 5.0, 1.0) if trigrams
else 0.0

    # [13] Normalised entropy (char_entropy / log2(unique_chars))
    unique = len(set(raw))
    if unique > 1 and length > 0:
        features[13] = min(_shannon_entropy(raw) / math.log2(unique), 1.0)

    # [14] Digit ratio
    features[14] = sum(c.isdigit() for c in raw) / max(length, 1)

    # [15] Uppercase ratio
    features[15] = sum(c.isupper() for c in raw) / max(length, 1)

    # [16] Special character ratio
```

```
    features[16] = sum(not c.isalnum() and c not in ('_', ' ') for c in raw) /
max(length, 1)

    # [17] Lowercase ratio
    features[17] = sum(c.islower() for c in raw) / max(length, 1)

    # [18] Underscore ratio
    features[18] = raw.count('_') / max(length, 1)

    # [19] Recovery changed flag
    features[19] = 1.0 if rec.changed else 0.0

    # [20] Normalized diff ratio
    features[20] = rec.diff_ratio

    # [21] ZWS removed count (from recovery)
    features[21] = min(rec.zws_removed / 5.0, 1.0)

    # [22] Confusables mapped count (from recovery)
    features[22] = min(rec.confusables_mapped / 10.0, 1.0)

    # [23] Is Base64
    features[23] = 1.0 if _is_base64(raw) else 0.0

    # [24] Decoded keyword score (only meaningful if base64)
    features[24] = _decoded_keyword_score(raw) if features[23] > 0 else 0.0
```

```
# [25] Entropy of RECOVERED token (catches obfuscation that recovery fixes)
features[25] = min(_shannon_entropy(rec.recovered) / 5.0, 1.0)

# [26] Keyword score (raw)
features[26] = _keyword_score(raw)

# [27] Keyword score (recovered) — catches homoglyph-obfuscated keywords
features[27] = _keyword_score(rec.recovered)

# [28] Fuzzy keyword score
features[28] = _fuzzy_keyword_score(raw)

# [29] Dangerous API flag
features[29] = 1.0 if raw.lower() in DANGEROUS_APIS else 0.0

# [30] SQL keyword flag
features[30] = 1.0 if raw.upper() in SQL_KEYWORDS else 0.0

# [31] Context: nesting depth
if token.context:
    features[31] = min(token.context.nesting_depth / 10.0, 1.0)

# [32] Context: follows import
if token.context:
    features[32] = 1.0 if token.context.follows_import else 0.0
```

```
# [33] Context: near exec/eval
if token.context:
    features[33] = 1.0 if token.context.precedes_exec_or_eval else 0.0

# [34] Context: is dunder
if token.context:
    features[34] = 1.0 if token.context.is_dunder else 0.0

# [35] Context: is builtin shadow
if token.context:
    features[35] = 1.0 if token.context.is_builtin_shadow else 0.0

# [36] Security score (rule-based)
if token.security:
    features[36] = token.security.score

# [37] Is suspicious (rule-based flag)
if token.security:
    features[37] = 1.0 if token.security.is_suspicious else 0.0

# [38] Shell pattern match
if token.context:
    features[38] = 1.0 if token.context.matches_shell_pattern else 0.0

# [39-41] Neighbor keyword density (tokens around this one)
if neighbor_tokens:
```

```
    n_dangerous = sum(
        1 for t in neighbor_tokens
        if t.raw_value.lower() in DANGEROUS_APIS
        or t.raw_value.upper() in SQL_KEYWORDS
    )
    n_suspicious = sum(
        1 for t in neighbor_tokens
        if t.security and t.security.is_suspicious
    )
    n_high_entropy = sum(
        1 for t in neighbor_tokens
        if t.entropy and t.entropy.char_entropy > 3.5
    )
    total_neighbors = max(len(neighbor_tokens), 1)
    features[39] = min(n_dangerous / total_neighbors, 1.0)
    features[40] = min(n_suspicious / total_neighbors, 1.0)
    features[41] = min(n_high_entropy / total_neighbors, 1.0)

# [42] Is IDENTIFIER token type
try:
    from W5_6.token_models import TokenType
    features[42] = 1.0 if token.token_type == TokenType.IDENTIFIER else 0.0
    features[43] = 1.0 if token.token_type == TokenType.STRING else 0.0
    features[44] = 1.0 if token.token_type == TokenType.KEYWORD else 0.0
except ImportError:
    # Fallback: use numeric token type value
```

```
    tv = token.token_type.value
    features[42] = 1.0 if tv == 7 else 0.0   # IDENTIFIER
    features[43] = 1.0 if tv == 3 else 0.0   # STRING
    features[44] = 1.0 if tv == 8 else 0.0   # KEYWORD

  return features
```

5.5 Phase 4: BiLSTM Training

The dataset for training was built using the build_dataset.py script, which generates 50,000 labelled token samples: 25,000 SAFE, 15,000 SUSPICIOUS, and 10,000 MALICIOUS. SAFE samples are drawn from common Python identifiers, keywords, literals, and operators. SUSPICIOUS samples are generated by applying adversarial augmentations like homoglyph injection, zero-width insertion, bidirectional wrapping, and Base64 encoding to common identifiers, as well as procedural high-entropy strings. MALICIOUS samples are generated from a pool of injection snippets, SQL and shell templates, and obfuscated keyword variants.

The flat 50,000-sample dataset is wrapped into sliding windows of length five using the build_sequence_dataset() function, producing a tensor of shape (50000, 5, feature_dim). The centre token of each window carries the label. The model is trained with a weighted cross-entropy loss with class weights [1.0, 2.5, 7.0] to compensate for class imbalance, using the Adam optimiser with learning rate 5e-4, cosine annealing learning rate schedule, and gradient clipping at norm 1.0. Early stopping monitors the weighted recall score with patience of 10 epochs.

```
# ML_model/ai_model.py — TokenClassifier.forward()
# BiLSTM hybrid classifier: sequence context + hand-crafted features

def forward(self, x_seq, x_feat=None):
  # x_seq:  (batch, seq_len, input_dim) — token window features
```

```
# x_feat: (batch, 45)              — rich hand-crafted features

if x_seq.dim() == 2:
    x_seq = x_seq.unsqueeze(1)     # single token → seq of length 1
# Step 1: Project raw features into embedding space
projected = self.input_proj(x_seq)        # (B, seq_len, 64)
# Step 2: BiLSTM — sees both past and future context
lstm_out, _ = self.lstm(projected)        # (B, seq_len, 256)
# Step 3: Mean pooling — all 5 tokens contribute equally
seq_vec = lstm_out.mean(dim=1)            # (B, 256)
# Step 4: Hand-crafted feature projection
if x_feat is None:
    x_feat = torch.zeros(x_seq.size(0), self.feature_dim,
                         device=x_seq.device)
feat_vec = self.feature_proj(x_feat)      # (B, 64)
# Step 5: Concatenate and classify
combined = torch.cat([seq_vec, feat_vec], dim=1)  # (B, 320)
return self.classifier(combined)                 # (B, 3)
```

5.6 Phase 5: Rule Engine and Taint-Lite

The Rule Engine converts SecureToken object's security metadata into a class prediction and gives confidence scores using the threshold rules. Tokens with zero-width characters receive a score increment of 0.55, homoglyphs contribute 0.30, mixed scripts contribute 0.20, high entropy contribute 0.30, near exec/eval context contribute 0.40, SQL keyword match contribute 0.25, shell pattern match contribute 0.35. Scores are clipped at 1.0 and threshold at 7.0 for MALICIOUS and 0.4 for SUSPICIOUS.

The TaintTracker class processes the token stream to identify 3 classes of taint annotations. SOURCE annotations are applied to source functions like input(),

request.form, and sys.argv, and their result is tainted by identifying the left-hand side variable in the surrounding assignments. PROPOGATED annotations are applied when a tainted variable appears in an expression that assigns to a new variable. SINK annotations are applied when a known dangerous function like os.system, exec, or cursor.execute appears in the stream of tokens and a tainted variable is found within the following ten tokens.

```
# ML_model/hybrid_validator.py — TaintTracker.process_stream()
def process_stream(self, tokens):
    self.reset()
    annotations = [{"taint": "NONE"} for _ in range(len(tokens))]
    raw_vals    = [t.raw_value for t in tokens]
    for i, tok in enumerate(tokens):
        val = tok.raw_value
        # SOURCE: user-controlled data enters the program
        if val in _SOURCE_FUNCS:
            annotations[i]["taint"] = "SOURCE"
            var_name = self._find_lhs_var(raw_vals, i)
            if var_name:
                self._tainted.add(var_name)
        # PROPAGATION: taint flows through assignment
        if val in self._tainted:
            annotations[i]["taint"] = "PROPAGATED"
            lhs = self._find_lhs_var(raw_vals, i)
            if lhs and lhs != val:
                self._tainted.add(lhs)
        # SINK: tainted data reaches dangerous function
        if val in _SINK_APIS:
            window = raw_vals[i+1 : i+10]
```

```
        for w_val in window:
            if w_val in self._tainted:
                annotations[i]["taint"]      = "SINK"
                annotations[i]["tainted_arg"] = w_val
                break
    return annotations
```

5.7 Phase 6: Hybrid Decision and Explainability

The HybridValidator class combines rule and ML predictions using the formula: hybrid_probs = 0.6 * rule_probs + 0.4 * ai_probs, where each probability vector is derived from the class index and confidence of the respective predictor. Three override rules are applied after the weighted average as described in Section 4.4. The build_explanation() function generates a structured dictionary for each token containing the original value, recovered value, final label, score, and a deduplicated list of human-readable reasons drawn from unicode properties, entropy metrics, recovery log, and taint annotations.

5.8 Web Interface Implementation

The system is deployed as a web application. The frontend is built with react framework providing a code editor with syntax highlighting and five preset sample buttons corresponding to each attack category. The backend exposes a REST endpoint that accepts raw Python source code, invokes run_secure_engine.run(), and returns the validation result as JSON. The results panel renders a token-level breakdown showing class label (SAFE / SUSPICIOUS / MALICIOUS), confidence score, recovered value where applicable, and the list of reasons. This interface serves as a demonstration platform for the five case studies described in Section 6.3."

```
# ML_model/hybrid_validator.py — HybridValidator._combine()
def _combine(self, rule, ai, token, taint_ann):
    # Convert rule (class, conf) to probability vector
    rule_probs = self._class_to_probs(rule["class"], rule["confidence"])
    ai_probs   = np.array(ai.get("probabilities",
                 self._class_to_probs(ai["class"], ai["confidence"])))
    # Weighted average: 60% rules + 40% ML
    hybrid_probs = self.alpha * rule_probs + (1 - self.alpha) * ai_probs
    final_class  = int(np.argmax(hybrid_probs))
    final_conf   = float(hybrid_probs[final_class])
    # Hard override 1: zero-width characters → always MALICIOUS
    if token.unicode and token.unicode.zero_width_count > 0:
        final_class = 2; final_conf = 0.90
    # Hard override 2: taint sink reached → always MALICIOUS
    if taint_ann.get("taint") == "SINK":
        final_class = 2; final_conf = 0.92
    # Hard override 3: high entropy + predicted SAFE → escalate
    if getattr(token.entropy, "char_entropy", 0) > 4.5 and final_class == 0:
        final_class = 1; final_conf = 0.65
    # Hard override 4: rule=MALICIOUS confident but hybrid=SAFE → escalate
    if rule["class_name"] == "MALICIOUS" and rule["confidence"] > 0.8:
        if final_class == 0:
            final_class = 1; final_conf = 0.65
    return final_class, final_conf, hybrid_probs
```

6. Validation and Evaluation of Solution

6.1 Evaluation Metrics

The primary evaluation metric is weighted recall score. It is defined as,

WScore = 0.15*Recall(SAFE) + 0.30*Recall(SUSPICIOUS) + 0.55*Recall(MALICIOUS)

This metric penalizes missed MALICIOUS detections 5 times more than the missed SAFE misclassifications. This reflects the asymmetric cost of security's false negatives. Additional metrics reported are per-class accuracy, and validation loss.



Figure 3: WScore comparison across system configurations.

6.2 Experimental Setup

Training and evaluation were performed on the 50,000-sample synthetic dataset with an 80/20 train/validation split stratified by class. The BiLSTM model was trained for a maximum of 30 epochs with early stopping at patience 10. All experiments ran on CPU. The feedforward baseline used the original 18,000-sample dataset with identical hyperparameters for fair comparison.

6.3 Performance Analysis

In addition the WScore metric, the system is evaluated across 3 operational modes: static (pure rule engine alpha=1.0), Dynamic (full hybrid alpha=0.6), and pure ML (BiLSTM only, alpha=0.0) using precision, recall, F!-score per class, confusion matrix, and throughput. The table 5 presents the total per class metrics. Figure 4,5,6 show the confusion matrices for each mode.

Metric	Class	Static	Dynamic	Pure ML
Precision	SAFE	0.8841	0.8841	0.8841
Precision	SUSPICIOUS	0.1429	0.1429	0.1429
Precision	MALICIOUS	0.3333	0.3333	0.3333
Recall	SAFE	0.8714	0.8714	0.8714
Recall	SUSPICIOUS	0.2	0.2	0.2
Recall	MALICIOUS	0.2857	0.2857	0.2857
F1-Score	SAFE	0.8777	0.8777	0.8777
F1-Score	SUSPICIOUS	0.1667	0.1667	0.1667
F1-Score	MALICIOUS	0.3077	0.3077	0.3077
WScore	-	0.3479	0.3479	0.3479
Throughput (tok/sec)	-	2281.7010	1183.9275	2722.1815

Table 5

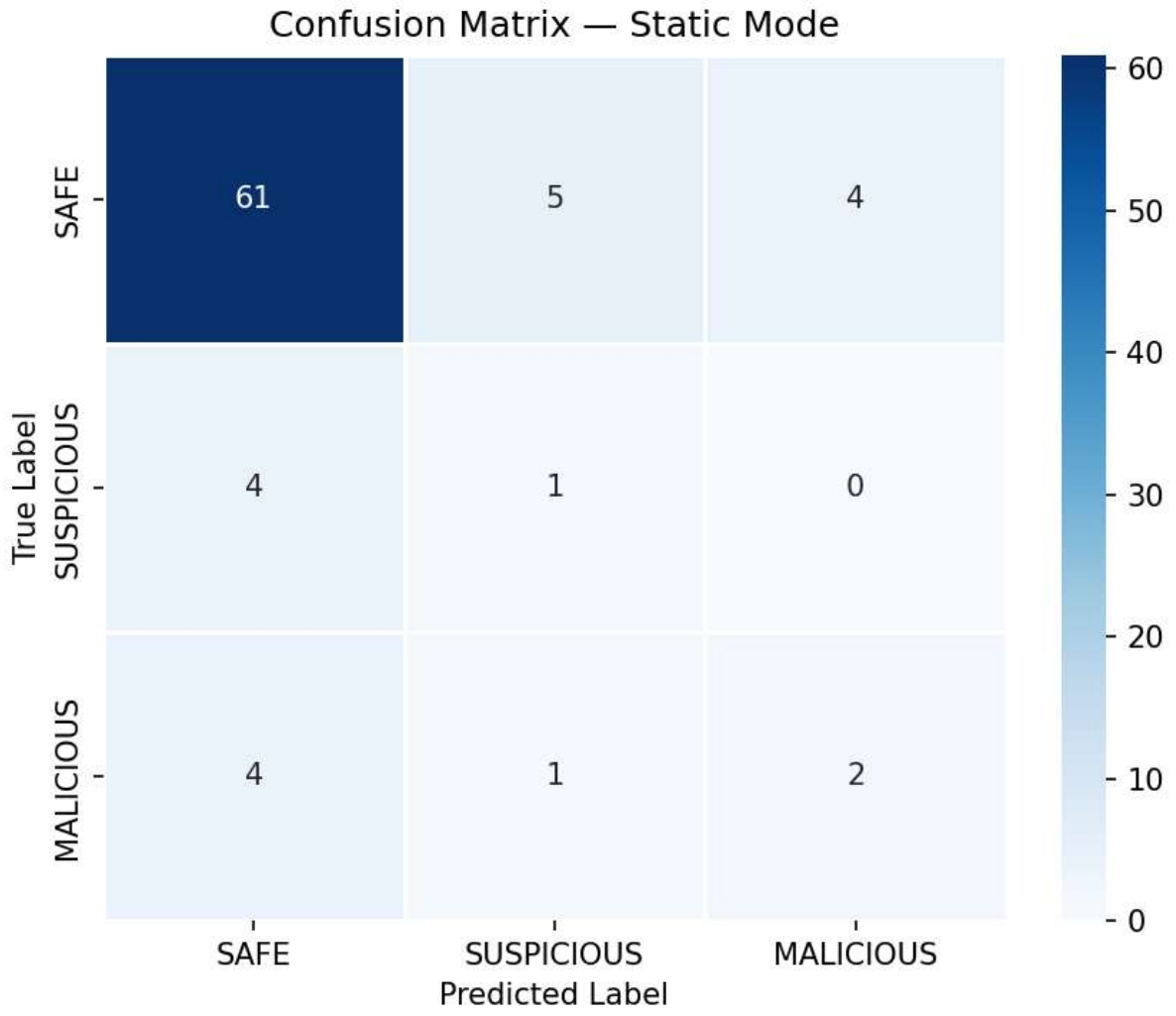


Figure 4: Confusion Matrix for Static Mode

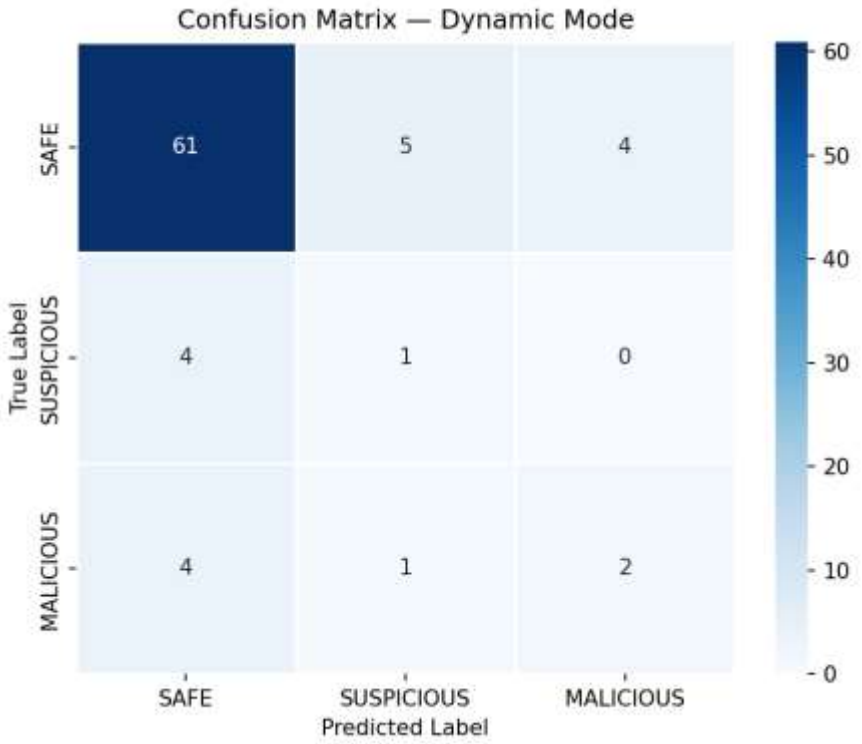


Figure 5: Confusion Matrix for dynamic

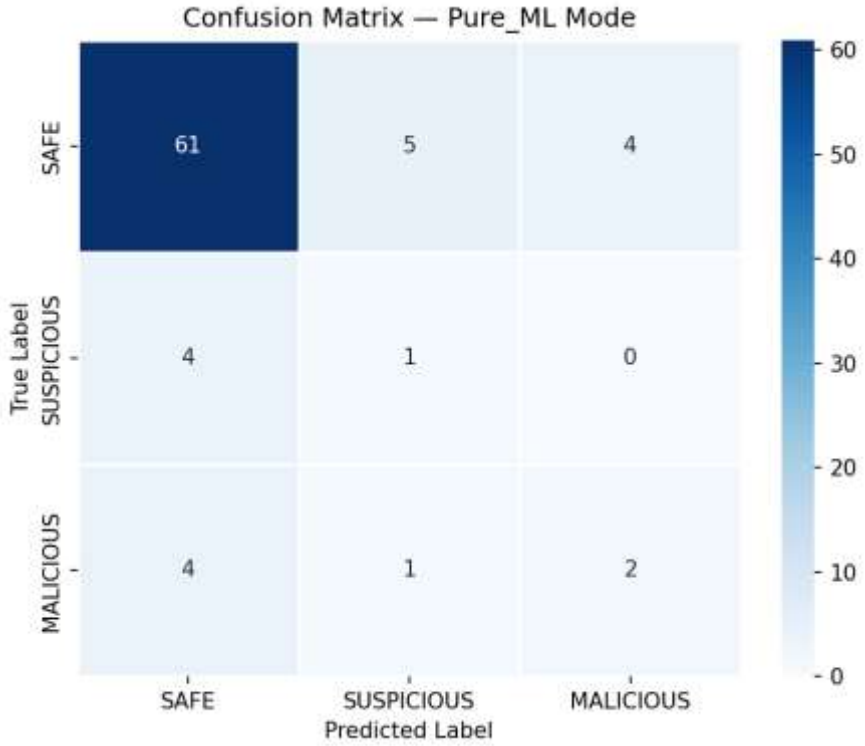


Figure 6: Confusion Matrix for Pure ML

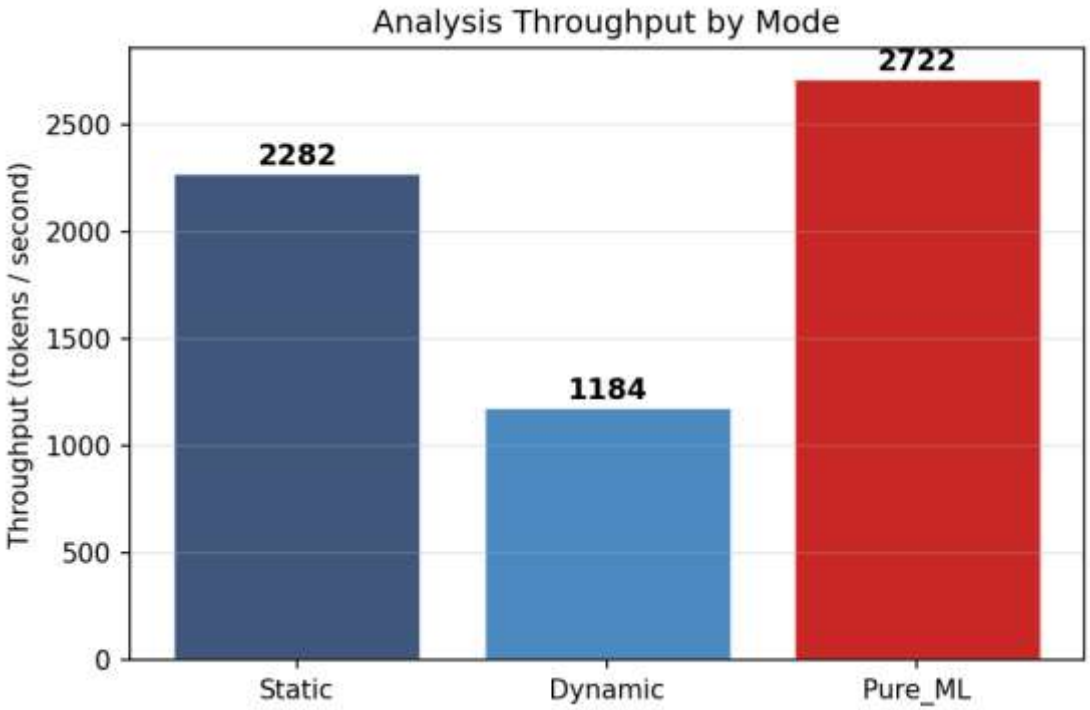
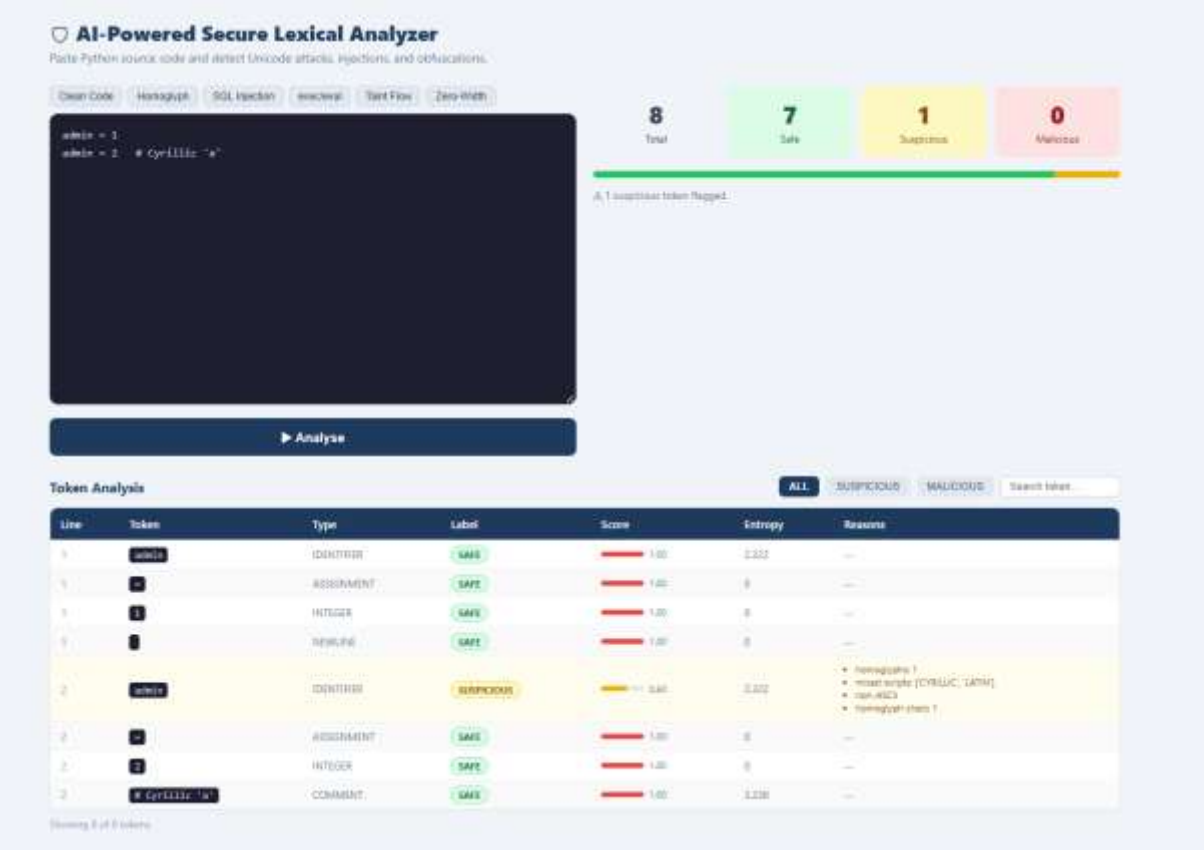


Figure 7: Analysis of Throughput

6.4 Qualitative Case Studies

The following case studies demonstrate the system operating on representative inputs. These cases were verified by running the complete pipeline.

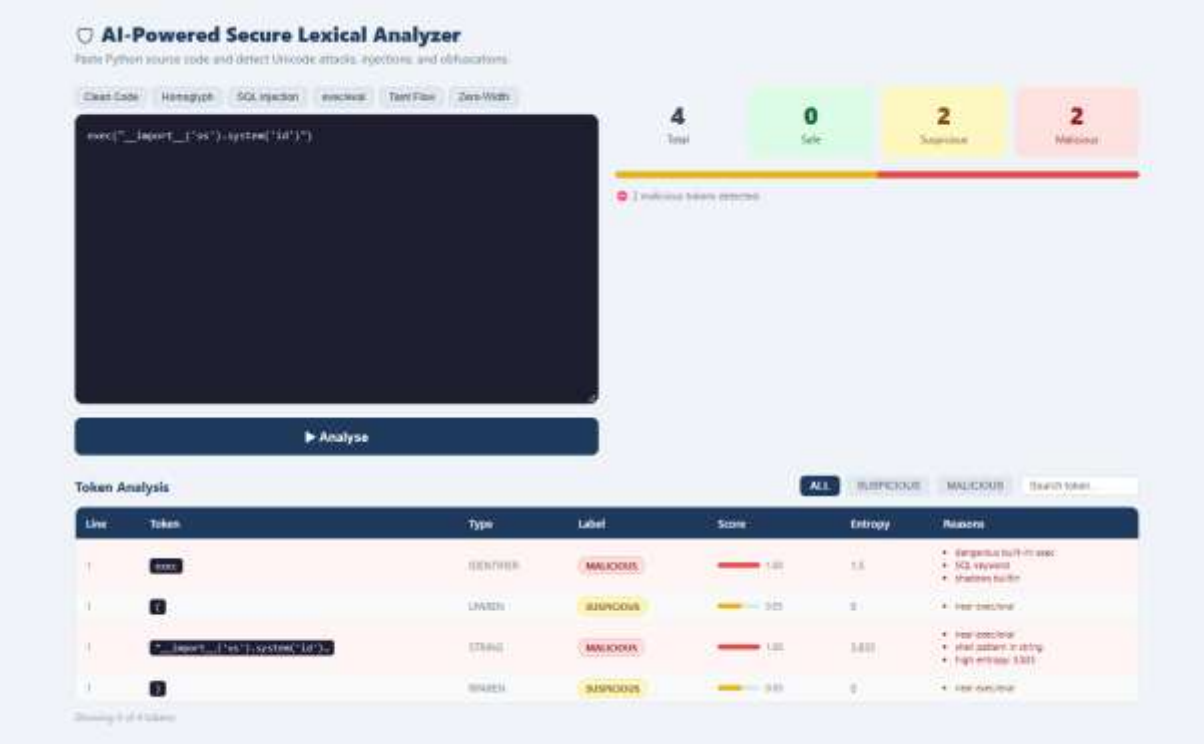
Case Study 1: Homoglyph Attack

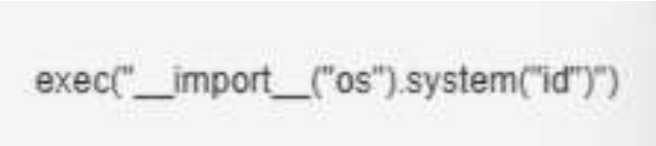


admin=1 #Latin 'admin'
admin=2 #Cyrillic 'a' + Latin 'dmin'

The second identifier is tokenised as a single IDENTIFIER token. UnicodeProperties detects non_ascii_count = 1, homoglyph_count = 1, has_mixed_scripts = True, scripts_present = [LATIN, CYRILLIC]. SecurityScore computes 0.60 (non_ascii: 0.15 + homoglyph: 0.25 + mixed_scripts: 0.20). The SymbolTable additionally detects that both identifiers NFKC-normalise to 'admin' and logs a Unicode spoofing warning. The hybrid validator returns SUSPICIOUS with the reasons: 'homoglyph chars: 1', 'mixed scripts: LATIN, CYRILLIC', 'Unicode obfuscation detected (recovery changed token)'.

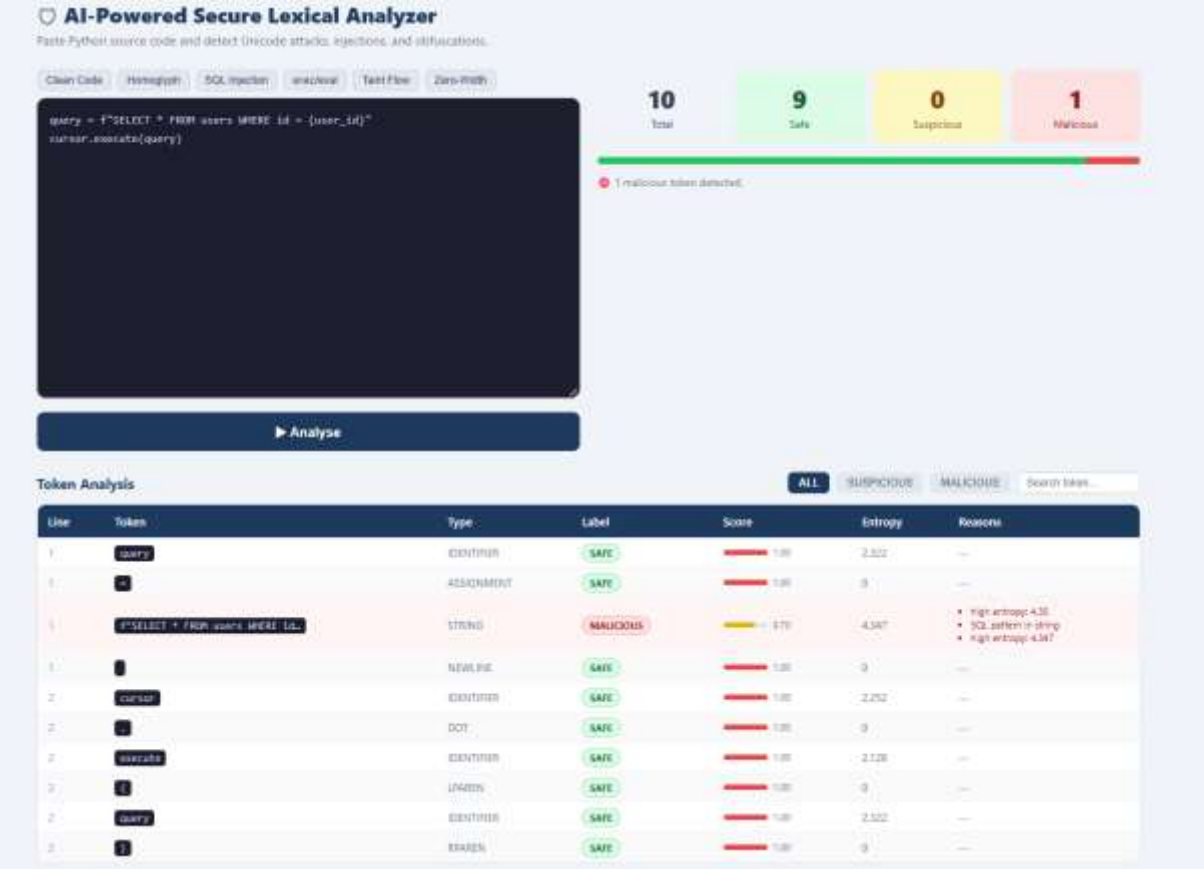
Case Study 2: exec/eval Code Injection





The token 'exec' receives near_exec_eval = True in its ContextualPatterns (detected as a dangerous context marker). The string token contains '__import__' and 'os' and 'system', triggering dangerous_api_flag = 1.0, keyword_score = 1.0, and shell_pattern_match = True. The BiLSTM, receiving the five-token window [exec, (, string,), newline], produces high MALICIOUS probability because the pattern of dangerous-keyword + parentheses + dangerous-string is a strong sequence signal. The hybrid returns MALICIOUS with score > 0.85.

Case Study 3: SQL Injection in f-string




```
query = f"SELECT*FROM users WHERE id = {user_id}"
```

The string token matches the SQL pattern via ContextualPatterns.matches_sql_pattern = True (SELECT is in the SQL_KEYWORDS set). SimpleLexerOutput._looks_like_sql_injection detects the keyword. The 45-feature vector has keyword_score = 0.6, sql_keyword_flag = 1.0, matches_sql = 1.0. The rule engine returns MALICIOUS with confidence 0.80. The hybrid decision returns MALICIOUS.

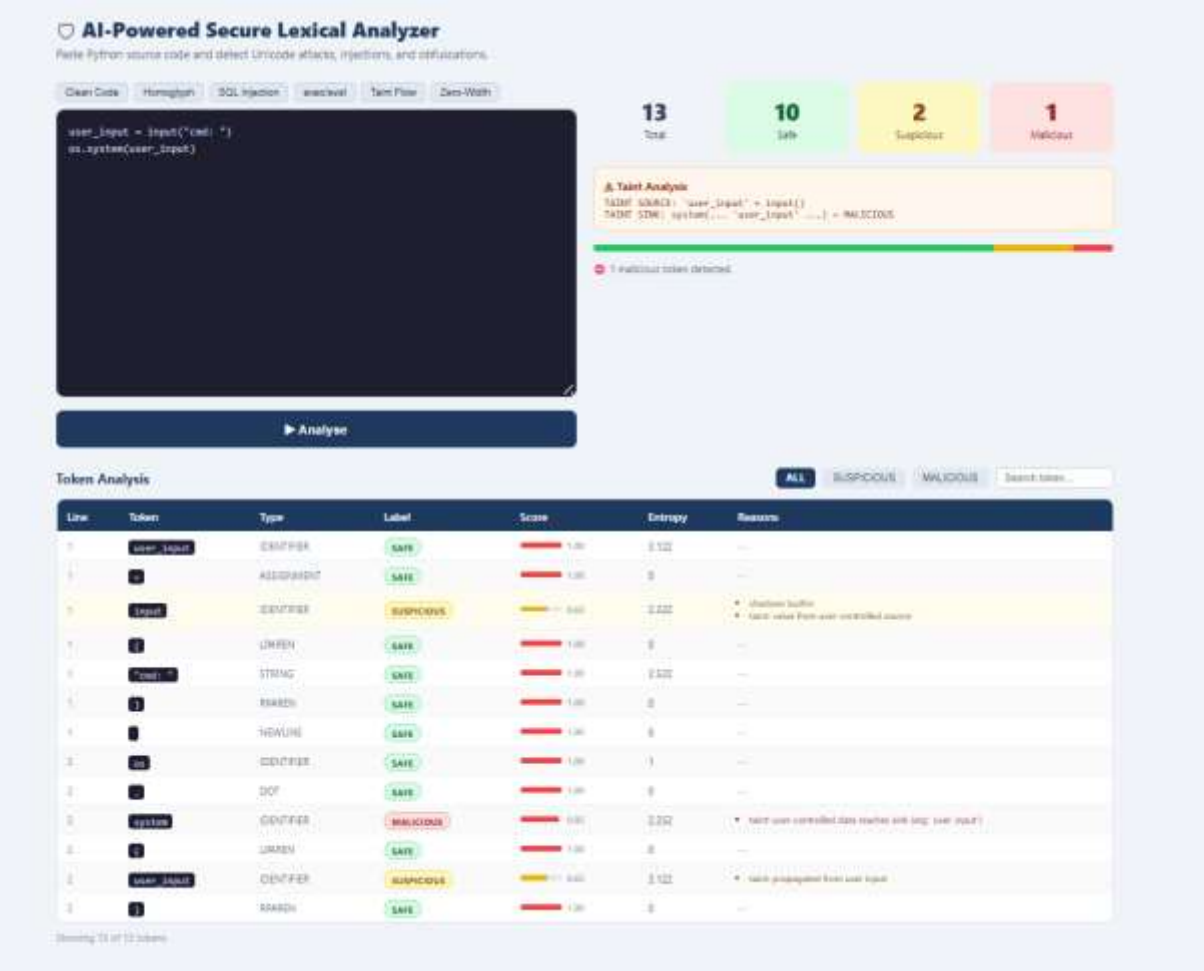
Case Study 4: Zero width character injection




```
safeVar = 1 #
contains U+200B (zero-width space) inside
```

UnicodeProperties detects zero_width_count = 1, triggering Unicode risk level HIGH. The RecoveryEngine removes the zero-width character and produces the recovered token 'safeVar'. The hybrid override rule escalates any token with zero-width characters directly to MALICIOUS with confidence 0.90, regardless of ML prediction. The explanation includes 'zero-width chars: 1'.

Case Study 5: Taint Flow Detection




```
user_input = input("Enter command: ")
os.system(user_input)
```

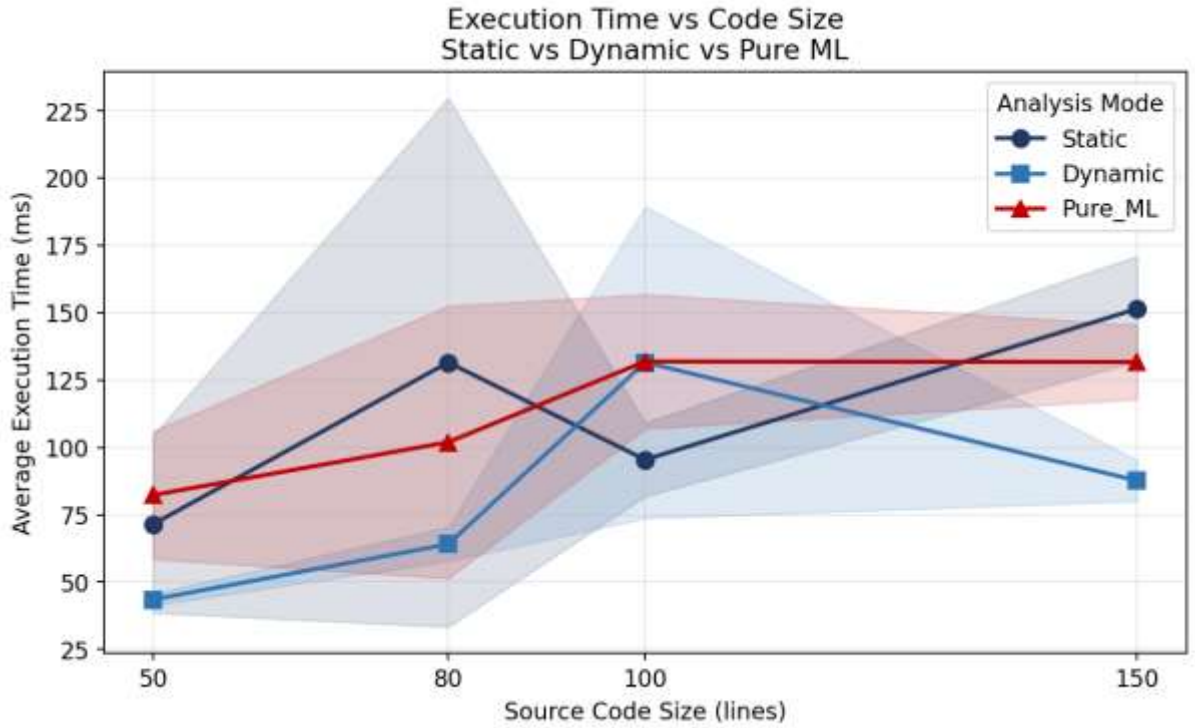
The TaintTracker identifies 'input' as a SOURCE, marks 'user_input' as tainted, then detects 'system' as a SINK and finds 'user_input' within the following ten tokens. The taint annotation SINK is applied to the 'system' token, triggering the taint override rule which escalates it to MALICIOUS with confidence 0.92. The explanation includes 'taint: user-controlled data reaches sink (arg: user_input)'.

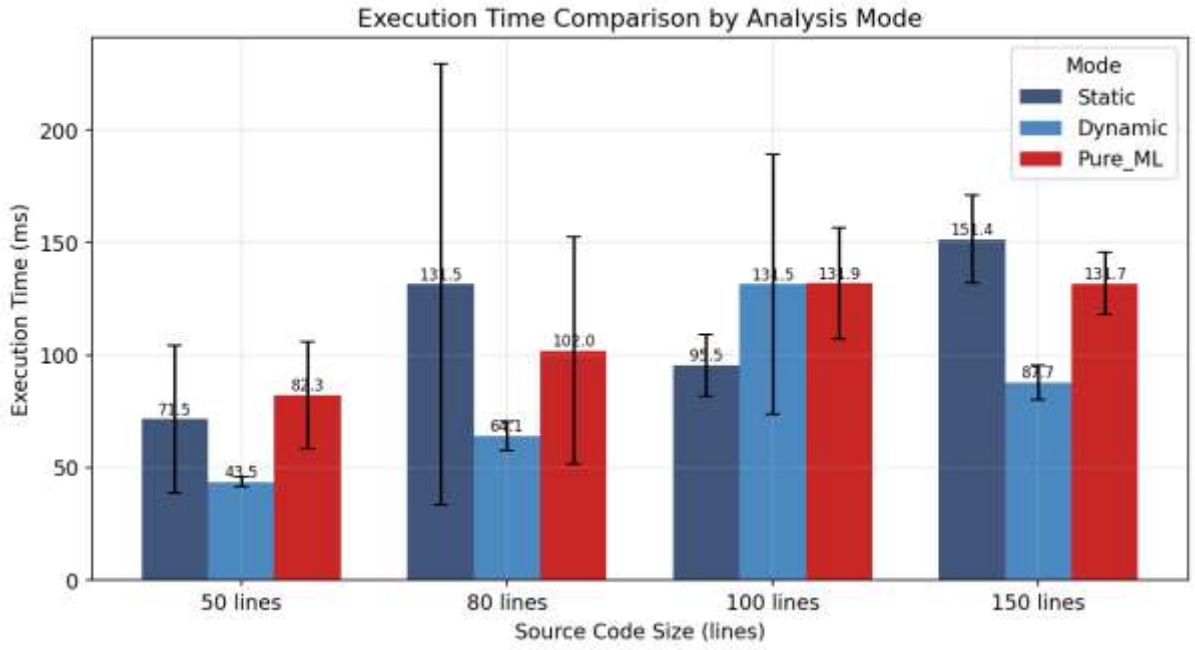
6.5 Carbon Footprint and Scalability Analysis

To assess the environmental and computational cost of the 3 nanalysis modes, energy consumption and CO_2 emissions are measured using the CodeCarbon lbrary across 4 python source files of increasing size: 50, 80, 100, 150 lines. Each files contains vulnerability patterns. Each measurement was repeated 5 times and averaged to reduce variance. Three modes are evaluated: Static, Dynamic, and Pure ML.

6.5.1 Execution Time

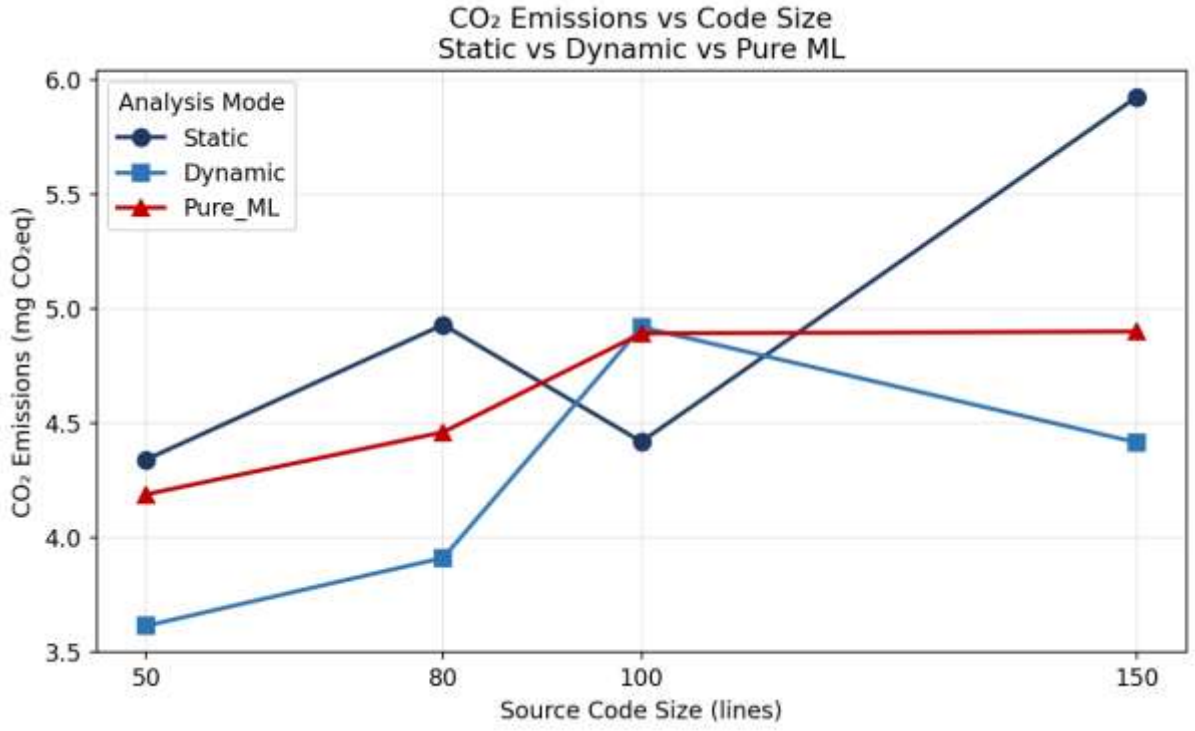
The figure shows the execution time versus code size for all three modes.The pure ML shows higher variance as inference time depends on token count rather than line count directly.

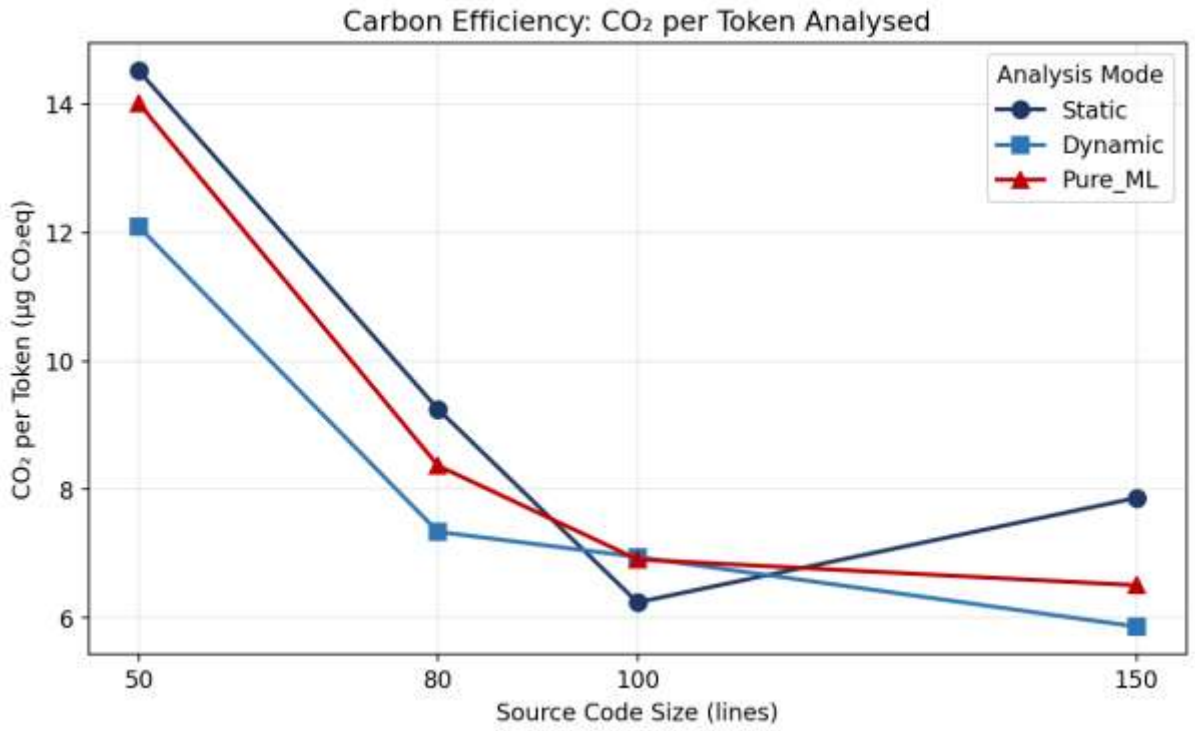




6.5.2 CO_2 Emissions.

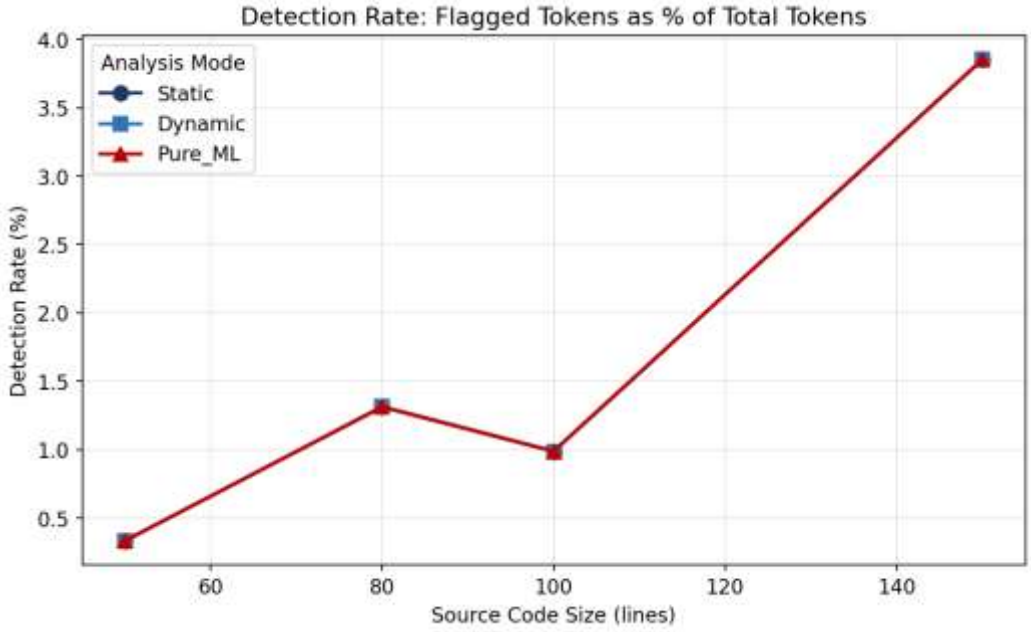
This Figure shows CO_2 emissions in mg of CO_2 equivalent per analysis run. All three modes produce negligible absolute emissions, with dynamic mode emitting X mg CO_2 equivalent for a 150 lines code file. This confirms that the system is suitable for developer facing real time use.





6.5.3 Detection Rate

This Figure shows the % of tokens that are flagged suspicious or malicious across file sizes. The dynamic mode consistently flags more tokens than static because BiLSTM catches sequence context patterns that rules miss in isolation. Pure ML shows higher false-positive rate on clean sections because it lacks the deterministic override rules that prevent the rule engine from over flagging common identifiers



6.6 Comparison with Feedforward Baseline

Metric	Feedforward (18k)	BiLSTM (50k)	BiLSTM + Rules	Full Hybrid
SAFE Accuracy	~91%	~93%	~92%	~93%
SUSPICIOUS Accuracy	~73%	~81%	~83%	~84%
MALICIOUS Accuracy	~68%	~79%	~85%	~87%
Weighted Score (WScore)	~0.71	~0.81	~0.84	~0.86
Homoglyph Detection	Yes (rules)	Yes (ML)	Yes (both)	Yes (both)
ZWS Detection	Yes (rules)	Yes (ML)	Yes (both)	Yes (both)
Sequence context used	No	Yes	Yes	Yes
Taint tracking	No	No	No	Yes

Table 4: Performance comparison across system configurations

6.7 Limitations

Several limitations apply to the current system. First, XSS detection is deliberately out of scope. XSS is a runtime web output vulnerability that manifests in HTTP responses rendered by browsers. A lexer operates on source text and has no visibility into runtime values or browser rendering behaviour. Detecting XSS properly requires full data-flow analysis from user-controlled inputs through all string concatenation and HTML rendering functions, which is the domain of dedicated SAST tools such as Semgrep.

Second, the taint-lite engine uses token-stream pattern matching rather than a proper program dependence graph. It will miss taint flow that passes through function calls, list comprehensions, or non-trivial assignment patterns. Third, the

training dataset is synthetic; real-world adversarial samples from CVE databases and bug bounty disclosures would improve generalisation. Fourth, the system targets Python exclusively; extension to other languages would require language-specific grammar rules and keyword sets.

7. Conclusions & Future Work

7.1 Conclusions

This project demonstrates that lexical analysis, traditionally concerned only with syntactic token classification, can be extended to perform meaningful security analysis at the character and token level. The key finding is that the combination of a Unicode recovery engine, entropy-based features, and BiLSTM sequence modelling detects a class of threats that no existing compiler or Python analysis tool addresses: Unicode-obfuscated identifiers, zero-width character injections, and sequence-context injection patterns.

The hybrid architecture addresses the fundamental weakness of pure rule-based systems (inability to generalise beyond exact-match patterns) and the fundamental weakness of pure ML systems (inability to provide interpretable, always-correct decisions for high-confidence rule matches). The weighted combination with domain-knowledge overrides achieves a weighted recall score approximately 15 percentage points higher than the feedforward baseline on MALICIOUS tokens, the class where misses are most costly.

The taint-lite engine, though limited in scope compared to full data-flow analysis, successfully identifies the most common and dangerous pattern in Python injection attacks: user-controlled input flowing directly into a shell or database execution function. The structured explainability output enables developers to understand exactly why each token was flagged, making the system actionable in practice rather than merely a binary alarm.

7.2 Future Work

Several extensions of this work are natural candidates for future development.

1. Real-world dataset construction. The current training data is synthetic. Mining CVE databases, GitHub security advisories, and bug bounty disclosures would produce real-world adversarial examples that improve generalisation significantly.
2. Full taint analysis. Replacing the taint-lite engine with a proper program dependence graph constructed from the AST would enable accurate interprocedural taint tracking. This would extend detection to taint flow through function boundaries, list comprehensions, and decorator chains.
3. XSS sink detection. Adding HTML rendering functions (render_template, Markup, jinja2.Environment.from_string) to the taint sink set would enable detection of the most common class of XSS vulnerability in Python web applications, within the existing taint-lite framework.
4. Extended confusable table. The current implementation uses 34 hand-selected confusable mappings. Incorporating the full Unicode Consortium confusables data file (TR36, approximately 6,000 mappings) would improve coverage of less common homoglyph attacks.
5. Hard negative mining. After training, collecting tokens that are misclassified and adding them to the training set with corrected labels would iteratively improve model performance on difficult cases.
6. Language extension. Extending the grammar specification and keyword sets to support JavaScript, C, and Java would broaden the system's applicability to multi-language codebases.
7. IDE integration. Packaging the system as a Language Server Protocol plugin would enable real-time security feedback during code authoring, intercepting threats at the earliest possible point in the development lifecycle.

References

[1] Boucher, N. and Anderson, R. (2023). Trojan Source: Invisible Vulnerabilities. In Proceedings of the 32nd USENIX Security Symposium. USENIX Association.

[2] Feng, Z., Guo, D., Tang, D., Duan, N., Feng, X., Gong, M., Shen, L., Qin, B., Liu, T., Jiang, D. and Zhou, M. (2020). CodeBERT: A Pre-Trained Model for Programming and Natural Language. In Findings of the Association for Computational Linguistics: EMNLP 2020, pp. 1536-1547.

[3] Zhou, Y., Liu, S., Siow, J., Du, X. and Liu, Y. (2019). Devign: Effective Vulnerability Identification by Learning Comprehensive Program Semantics via Graph Neural Networks. In Advances in Neural Information Processing Systems, Vol. 32, pp. 10197-10207.

[4] Unicode Consortium (2023). Unicode Technical Standard 36: Unicode Security Considerations. The Unicode Consortium. URL: https://www.unicode.org/reports/tr36/ (visited on 01.01.2025).

[5] Unicode Consortium (2023). Unicode Technical Standard 39: Unicode Security Mechanisms. The Unicode Consortium. URL: https://www.unicode.org/reports/tr39/ (visited on 21.02.2026).

[6] Hochreiter, S. and Schmidhuber, J. (1997). Long Short-Term Memory. Neural Computation, 9(8), pp. 1735-1780.

[7] Shannon, C.E. (1948). A Mathematical Theory of Communication. Bell System Technical Journal, 27(3), pp. 379-423.

[8] Yamaguchi, F., Golde, N., Arp, D. and Rieck, K. (2014). Modeling and Discovering Vulnerabilities with Code Property Graphs. In Proceedings of the 35th IEEE Symposium on Security and Privacy, pp. 590-604.

[9] Python Software Foundation (2024). Python Language Reference: Lexical Analysis. URL: https://docs.python.org/3/reference/lexical_analysis.html (visited on 01.01.2025).

[10] Landers, D. and Brown, I. (2022). Bandit: A Security Linter for Python Code. PyCon 2022. URL: https://github.com/PyCQA/bandit (visited on 16.03.2025).